

计算机组成

MIPS体系结构 ～异常与中断

高小鹏

北京航空航天大学计算机学院

提纲

- 程序轮询I/O
- 异常/中断
- 支持异常/中断
- 协处理器
- 多任务切换机制初探

程序查询I/O

- 一般来说设备上都有2个寄存器
 - ◆ 状态寄存器（Status）：至少包含指示是否就绪的Ready位
 - 0: 未就绪（不可以输入/输出），1: 就绪（可以输入/输出）
 - ◆ 数据寄存器（Data）：用于传输数据
- 程序查询（Polling）I/O的基本原理：
 - ◆ 1) cpu循环的读取status寄存器，直到Ready置位（0→1）
 - ◆ 2) cpu用load指令（或store指令）读（或写）Data寄存器
 - 设备译码并执行完cpu的操作后，会复位Ready位（1→0）

程序查询I/O示例

- 输入：读取键盘值到\$V0

```
                lui    $t0, 0xffff # ffff0000, 基地址
Waitloop:      lw      $t1, 0($t0) # status 寄存器
                andi   $t1, $t1, 0x1
                beq    $t1, $zero, Waitloop
                lw      $v0, 4($t0) # data reg
```

- 输出：把\$a0写入设备

```
                lui    $t0, 0xffff # ffff0000, 基地址
Waitloop:      lw      $t1, 8($t0) # status 寄存器
                andi   $t1, $t1, 0x1
                beq    $t1, $zero, Waitloop
                sw      $a0, 12($t0) # data reg
```

常见I/O设备的性能参数

I/O设备间的性能差距巨大：LAN与鼠标有7个数量级的性能差距

设备	行为	人机交互	带宽(KB/s)
键盘	输入	是	0.01
鼠标	输入	是	0.02
音频输出	输出	是	5.00
打印机	输出	是	100.00
硬盘	存储	否	10,000.00
WLAN	输入/输出	否	10,000.00
显示	输出	是	30,000.00
千兆LAN	输入/输出	否	125,000.00

处理器/设备性能不匹配

在程序查询模式中，由于设备太慢，cpu大量执行低效的循环

- 假设cpu的主频为1GHz，则cpu可以每秒执行 10^9 条load/store指令
 - cpu：相当于带宽为4,000,000KB/s
 - I/O设备：带宽从0.01KB/s到125,000KB/s
- 输入：当cpu读取数据时，设备很难及时提供数据
 - 另外，很多时候，人机交互型设备还必须等待人的操作，例如键盘输入
- 输出：当cpu写数据时，设备很难及时接收并存储数据

为了提高性能，需要一种非计划型的过程

- 1) 当设备就绪后，设备向cpu发出一个信号
- 2) cpu在接收到该信号后，首先完成当前指令，然后自动去执行预先设置的特定过程
- 3) 该特定过程执行结束后，cpu再返回至被打断的位置恢复执行

提纲

- 程序轮询I/O
- 异常/中断
- 支持异常/中断
- 协处理器
- 多任务切换机制初探

中断（Interrupt）

- 中断：由I/O设备产生的异步事件
 - ◆ 异步：事件与当前指令流之间不存在关联关系（或因果关系）
- 产生原因：设备处于某种状态，需要cpu进行处理
 - ◆ 例如，准备接收或发送数据、某些特殊情况（如出错）
- 通知机制：来自设备的请求信号
 - ◆ 设备使该信号从无效变为有效来通知cpu
- 中断处理：cpu执行一段被称为中断服务程序（interrupt service routine, ISR）来处理设备请求
 - ◆ cpu接收到中断请求后，首先执行完当前指令，然后自动转入ISR
 - ◆ 执行完ISR后，cpu再返回到被中断的位置继续执行

异常 (Exception)

- 异常是由于cpu执行指令而产生的同步事件
- 产生原因：指令执行出现“不期望”状态，需要cpu进行处理
 - ◆ 例如：溢出、除0、TLB缺失等
- 通知机制：数据通路中的功能部件向控制器产生请求信号
 - ◆ 可以理解为cpu内部的中断请求信号
- 异常处理
 - ◆ 由于是指令执行出现了问题，因此cpu停止继续执行当前指令，类似中断处理一样转入特定的异常处理程序
 - ◆ 执行完异常处理程序后，cpu再重新执行出问题的指令

异常与中断的区别

- 虽然异常与中断的处理机制高度类似，但两者也具有明显的不同
- 1) 同步与异步
 - ◆ 异常：与指令流是同步关系（有因果关系）
 - ◆ 中断：与指令流是异步关系（无因果关系）
- 2) 受害指令（即发生异常或中断时的那条指令）
 - ◆ 异常：停止执行受害指令，然后转入异常处理程序
 - ◆ 中断：执行完受害指令，然后转入中断处理程序
- 3) PC保存
 - ◆ 异常：由于当前指令未执行完，因此当前指令地址保存到EPC
 - ◆ 中断：由于当前指令执行完了，因此下一条指令地址保存到EPC

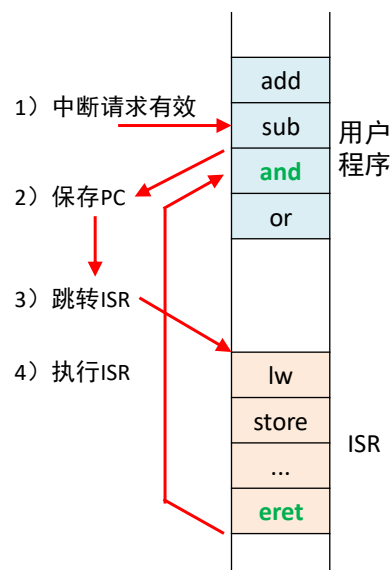
CPU中的硬件机制

- MIPS有一个名为系统控制协处理器(CP0)来管理中断/异常
- 保存被中断指令的下一条指令的地址
 - ◆ cpu要执行完当前指令才响应中断，因此要保存的值是PC+4
 - ◆ 该值被保存在EPC (Exception Program Counter) 中
- 固定的ISR首条指令地址 (即入口地址)
 - ◆ 由于cpu自动跳转至ISR，因此ISR入口地址必须是固定值，即BFC00000h

异常入口地址：不同系统不同，同一系统也可能不同

中断处理I/O的基本过程示例

- 1) 设备向cpu发出中断请求信号
- 2) cpu执行完sub后，将sub下一条指令的PC值保存好
- 3) cpu自动将ISR的首地址加载到PC中，从而实现跳转
- 4) cpu执行ISR完成输入输出等操作
- 5) cpu执行eret指令实现从ISR返回
 - ◆ eret功能之一是将保存的PC值写入PC



提纲

- 程序轮询I/O
- 异常/中断
- 支持异常/中断
- 协处理器
- 多任务切换机制初探

硬件：支持异常和中断^{1/4}

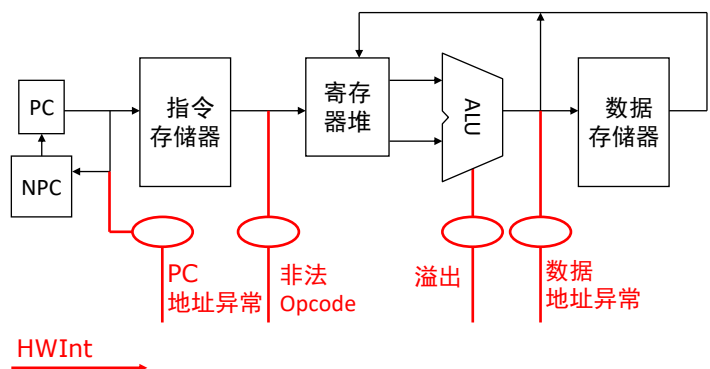
- 如何支持在不同阶段可能同时发生的多个异常？
 - ◆ PC地址出错、非法指令、算术溢出、数据地址出错

- 如何支持来自外部I/O设备的中断？

- ◆ HWInt[7:2]对应6个硬件中断

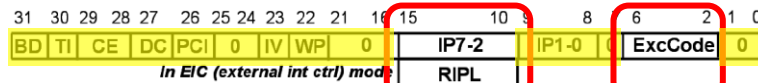
- 中断与异常，谁的优先级高？

- ◆ 异常高、中断低：通用CPU
- ◆ 中断高、异常低：嵌入式实时CPU



硬件：支持异常和中断^{2/5}

□ Cause寄存器：保存的是当前有效的中断请求、异常编码



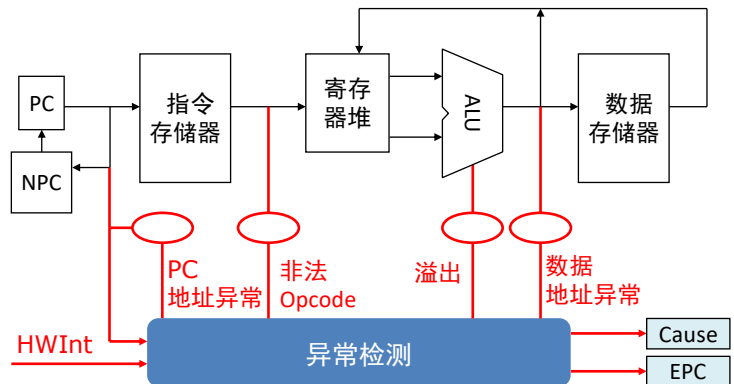
◆ IP[7:2]: 6位等待处理的设备中断

- 记录当前哪些硬件中断正在有效
- 1-有中断；0-无中断

◆ ExcCode[6:2]: 异常编码

- 共计32种
- 与课程相关的主要异常类型

ExcCode	助记符	描述
0	Int	中断
10	RI	不识别(非法)指令
12	Ov	算数指令导致的异常(如add)



15

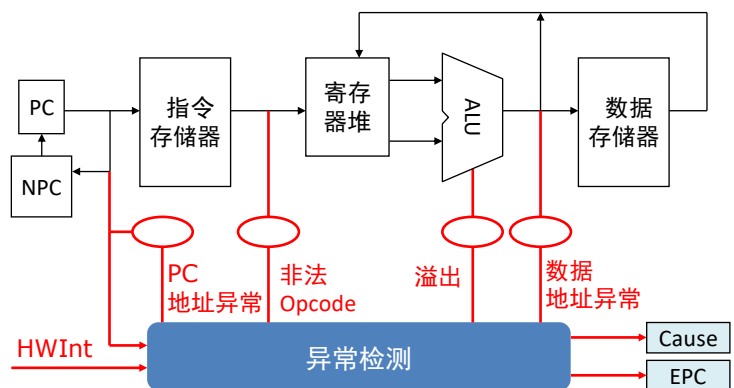
硬件：支持异常和中断^{3/5}

□ EPC寄存器：保存中断/异常时的PC

- ◆ 异常时保存的是PC（当前指令），中断时保存的是PC+4（下一条指令）

□ ERET指令：中断/异常服务程序返回指令

- ◆ PC ← EPC
- ◆ 用于从异常处理程序返回



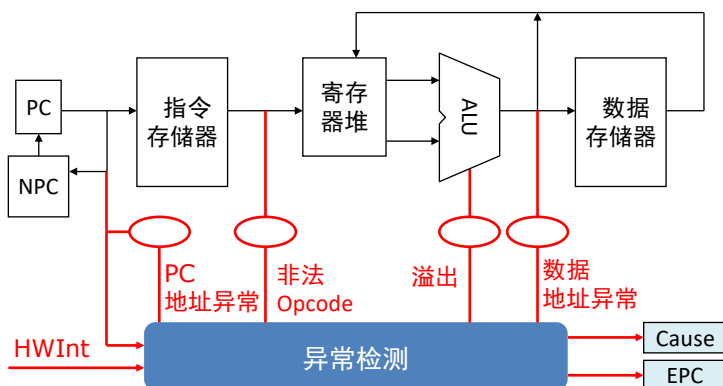
16

硬件：支持异常和中断^{4/5}

- Cause：保存的是当前有效的中断请求、异常编码
- EPC：异常时保存的是PC（当前指令），中断时保存的是PC+4（下一条指令）
- 异常检测：产生正确的异常编码、EPC值、NPC异常入口地址选择

中断与异常的优先级

- ◆ 指令（含异常）高于中断
- ◆ 前级异常先于高于后级异常



17

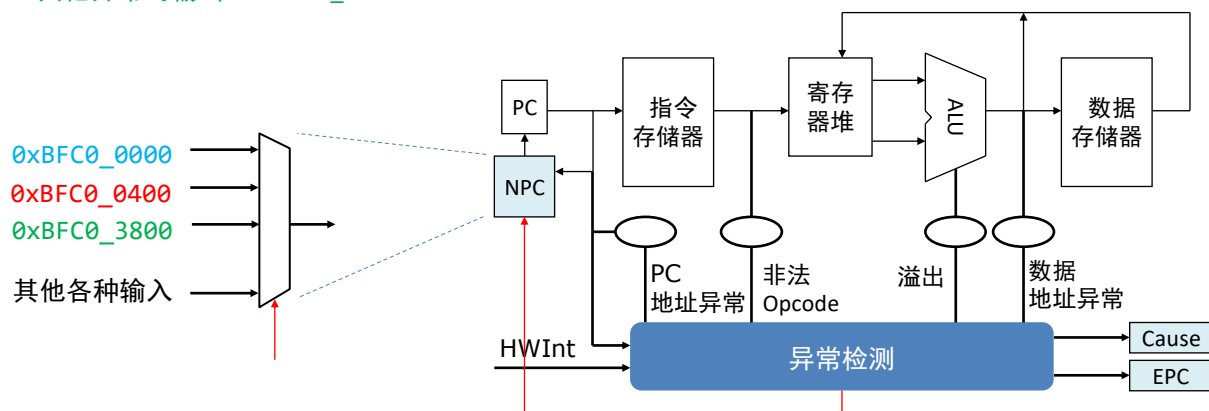
硬件：支持异常和中断^{5/5}

NPC：输出正确的异常入口地址

- ◆ 系统复位时输出：0xBFC0_0000
- ◆ 硬件中断时输出：0xBFC0_0400
- ◆ 其他异常时输出：0xBFC0_0380

MARS：2种地址

- ◆ default：0x8000_0000
- ◆ compact：0x0000_4180



18

软件：中断服务程序

□ 框架结构：保存现场、中断处理、恢复现场、中断返回

□ 1、保存现场

- ◆ 将所有寄存器都保存在堆栈中

□ 2、中断处理

- ◆ 读取特殊寄存器了解哪个硬件中断发生
- ◆ 执行对应的处理策略（例如读写设备寄存器、存储器等）

□ 3、恢复现场

- ◆ 从堆栈中恢复所有寄存器

□ 4、中断返回

- ◆ 执行`eret`指令

1、3、4：通用代码
2：针对特定设备

中断/异常响应机制：控制器

□ ①保存：将正确的PC值和ExcCode保存在EPC和Cause中

□ ②跳转：产生相应的中断处理程序入口地址，并写入PC

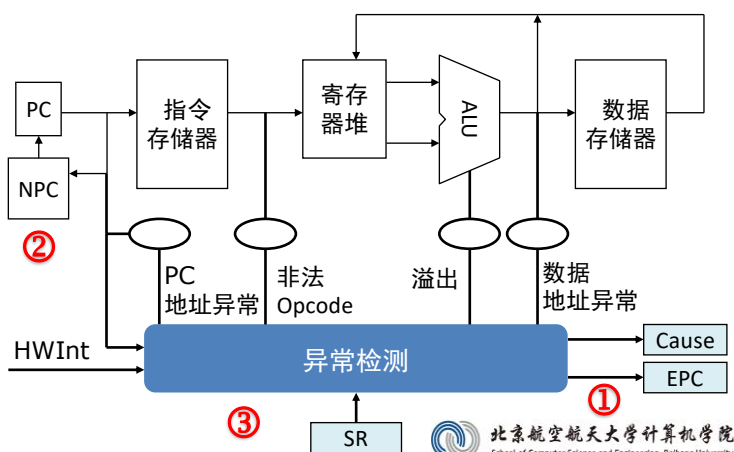
□ ③关中断：EXL置位，防止再次进入

□ 实现要点

- ◆ 3个步骤在**同一周期完成**

□ 实现方法

- ◆ EPC/Cause/PC/SR写使能**同时产生**



中断响应机制：ERET指令

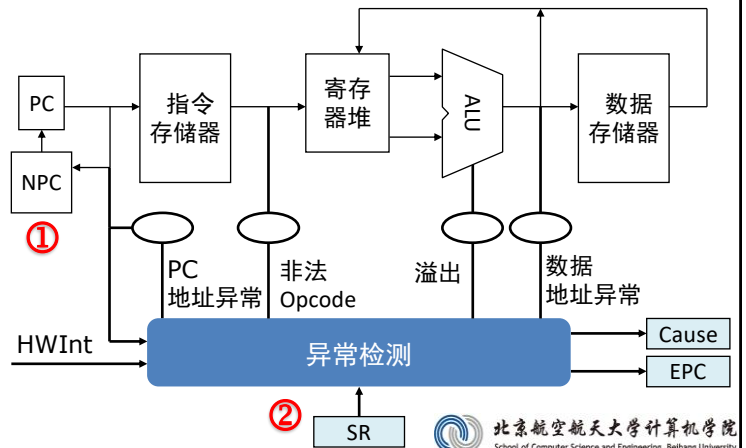
- ①恢复PC：将EPC写入PC
- ②开中断：清除EXL，允许再次响应中断

实现要点

- 2个步骤在**同一周期**完成

实现技巧

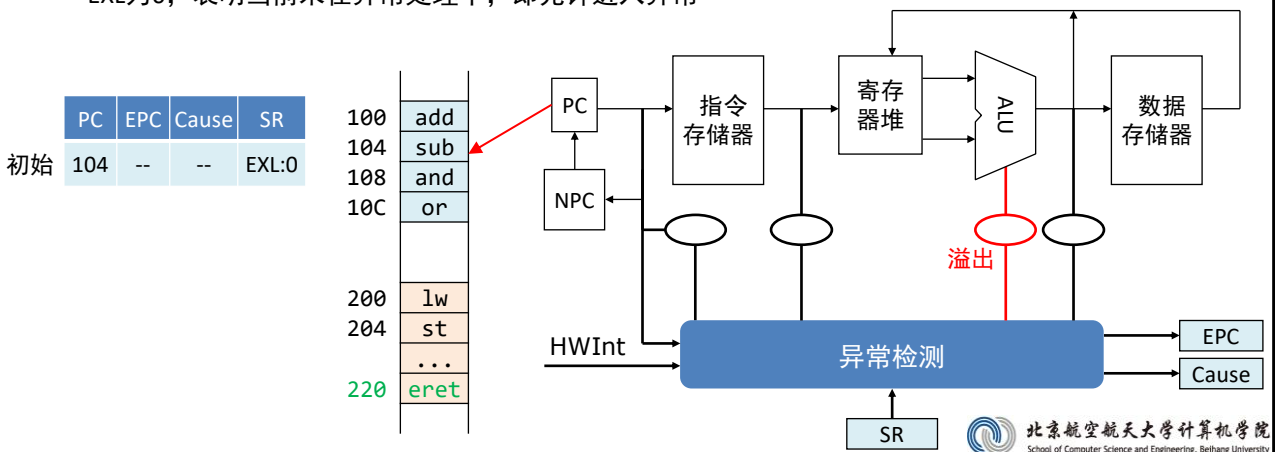
- PC写使能/EXL清除**同时产生**



中断/异常软硬件协同响应机制：异常^{1/4}

- 假设：sub指令执行过程中产生了算术异常
- 初始：PC指向sub，ALU产生溢出信号
 - EPC和Cause的值是无意义的
 - EXL为0，表明当前未在异常处理中，即允许进入异常

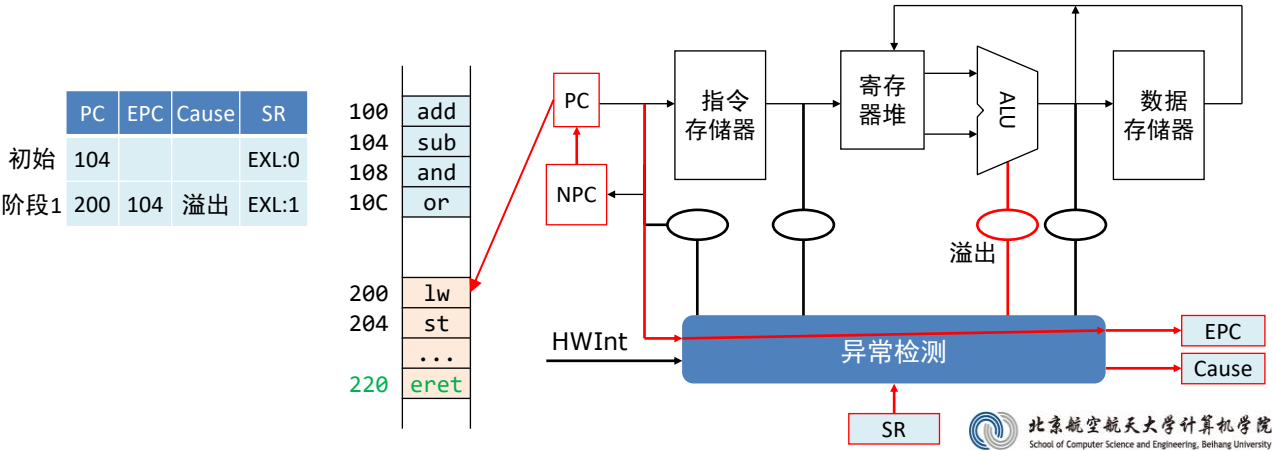
异常检测机制捕捉到异常发生，于是系统进入异常响应的三阶段



中断/异常软硬件协同响应机制：异常^{2/4}

阶段1（硬件）：下述3个操作必须在同一个cycle里完成

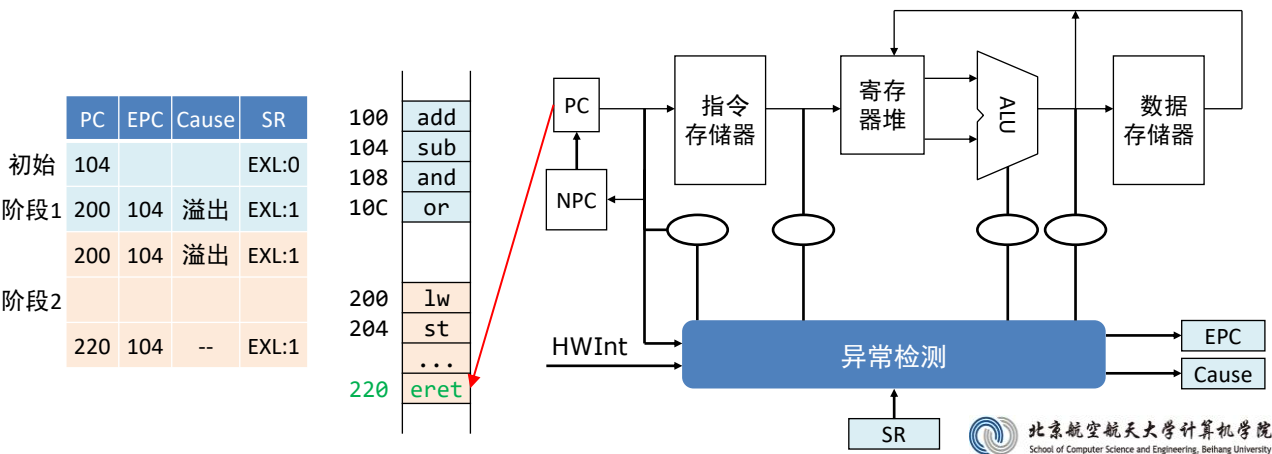
- ① $EPC \leftarrow PC$, $Cause \leftarrow$ 溢出编码
- ② $PC \leftarrow$ 异常地址
- ③ $EXL \leftarrow 1$ （表明进入异常处理，即防止了重入异常）



中断/异常软硬件协同响应机制：异常^{3/4}

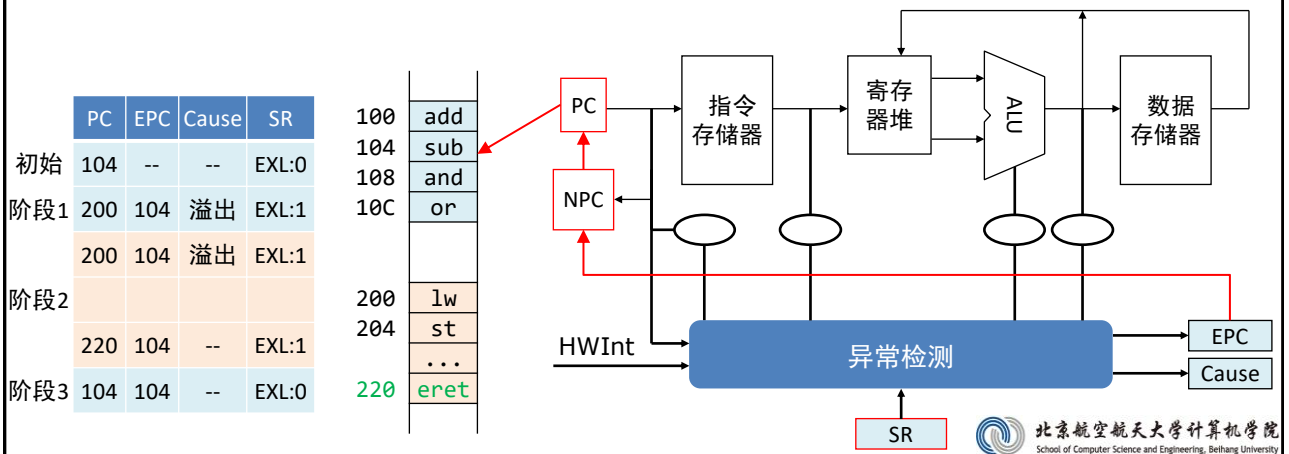
阶段2（软件）：执行异常服务程序

- 异常服务程序的最后一条指令为 **eret** 指令



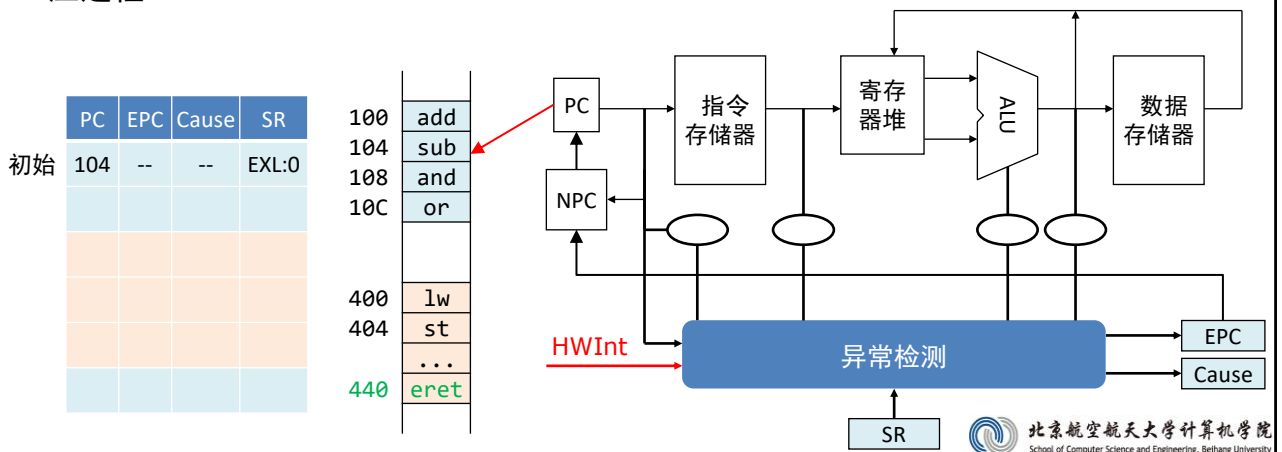
中断/异常软硬件协同响应机制：异常^{4/4}

- ◆ ①PC←EPC
- ◆ ②EXL←0（退出异常处理，即允许再次响应异常）



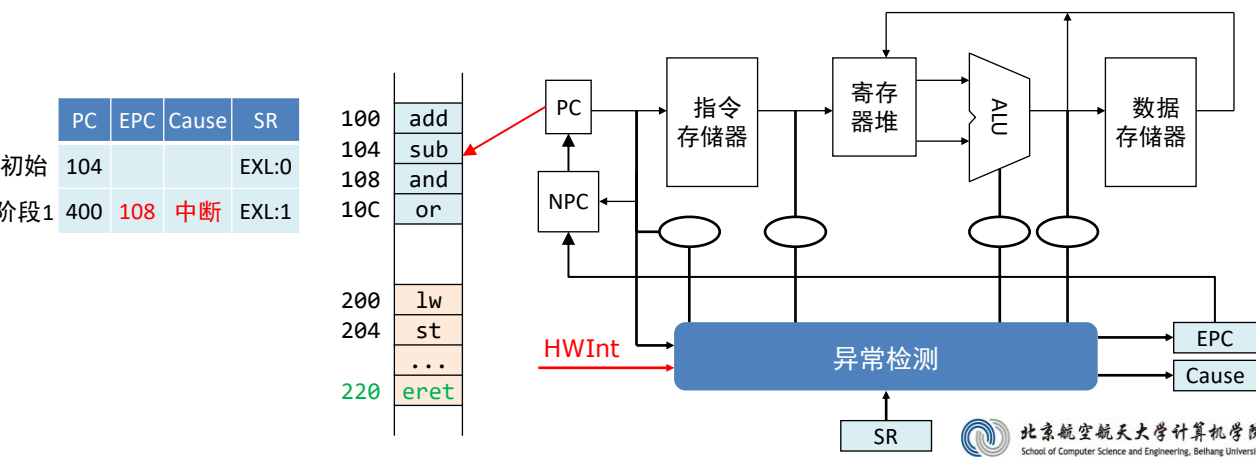
中断/异常软硬件协同响应机制：中断^{1/2}

- 假设：sub指令执行过程中，某个外部设备产生了中断请求
- 中断与异常的处理框架基本相同。区别在于中断要等当前指令执行结束，才能开始响应过程



中断/异常软硬件协同响应机制：中断^{2/2}

- 具体差别：sub执行是否结束、EPC值
 - 异常：sub不能执行完（RFWr=0），EPC保存sub的地址
 - 中断：sub必须执行完（RFWr=1），EPC保存and的地址



提纲

- 程序轮询I/O
- 异常/中断
- 支持异常/中断
- 协处理器
- 多任务切换机制初探

CP0概述

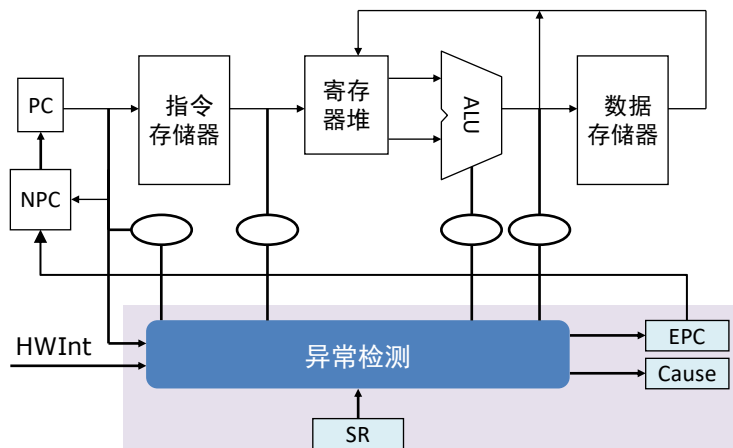
4个寄存器：SR、Cause、EPC、PRId

- 阅读《See MIPS Run Linux》第3章
- 无关寄存器及无关位可以不阅读

指令：MFC0、MTC0

- MFC0：读取CP0寄存器，写入通用寄存器
- MTC0：读取通用寄存器，写入CP0寄存器

寄存器号	寄存器
12	SR
13	CAUSE
14	EPC
15	PRId



设计CP0：SR

由于无用位较多，因此只定义有用位

- `reg [15:10] im ;`
- `reg exl, ie ;`

`reg [5:0] im`与`reg [15:10] im`等价的，但后者可读性更好

SR整体表示为：{16'b0, im, 8'b0, exl, ie}

im, ie：行为很简单

```
if (当Wen有效 && Sel为对应的寄存器编号)
    {im, exl, ie} <= {DIn[15:10], DIn[1], DIn[0]} ;
```

exl：除了类似im/ie的行为外，还必须有置位和清除的功能。以置位为例：

```
if (EXLSet)
    exl <= 1'b1 ;
```

设计CP0: Cause

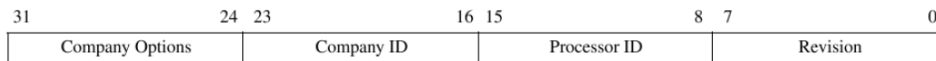
- Cause: 只需定义6位寄存器, 在clock上升沿不断的保存外部6个中断(HWInt[5:0])
 - ◆ `reg [15:10] hwint_pend ;`
- Cause整体表示为:
 - ◆ `{16'b0, hwint_pend, 10'b0}`

设计CP0: EPC

- 定义30位寄存器
 - ◆ `reg [32:2] epc;`
- 为什么不需要32位?

设计CP0: PRId

- 用于对公司/指令集版本等进行标识
 - ◆ Intel处理器也有ID, CPU-Z就可以读取



- 目前可以任意选则用一个4字节的编码值, 如
 - ◆ 0x1234_5678

设计CP0: 输出CP0寄存器

- 除了SR/Cause/EPC/PRId外, 不用的寄存器一律输出0
- 可以设计5选1的MUX, 或者用行为描述 (如样例代码)

```
assign DOut = (Sel==12) ? {16'b0, im, 8'b0, exl, ie} :  
              (Sel==13) ? {xxxxxxxxxxxxxxxxxxxxxxxxxxxx} :  
              (Sel==14) ? EPC :  
              (Sel==15) ? PrID  
                      32'b0 ;
```

异常检测

变量：全局请求int_exc，设备中断hwint，异常请求exc_ov...

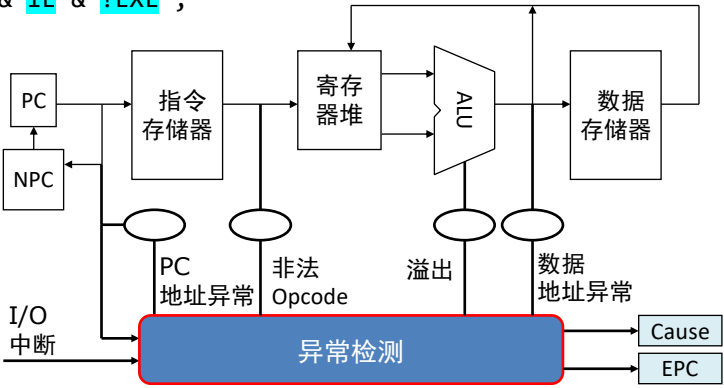
```
hwint    = |(HWInt[7:2] & IM[7:2])|
exc_pc   = !(PC ∈ 代码段)
exc_ov   = ...
.....
int_exc  = (hwint | exc_ov | .....) & IE & !EXL ;
```

变量：Cause的ExcCode[6:2]

```
exc_code = hwint ? 00 :
            exc_pc ? xx :
            .....
```

变量：pc_exc（现场对应的PC值）

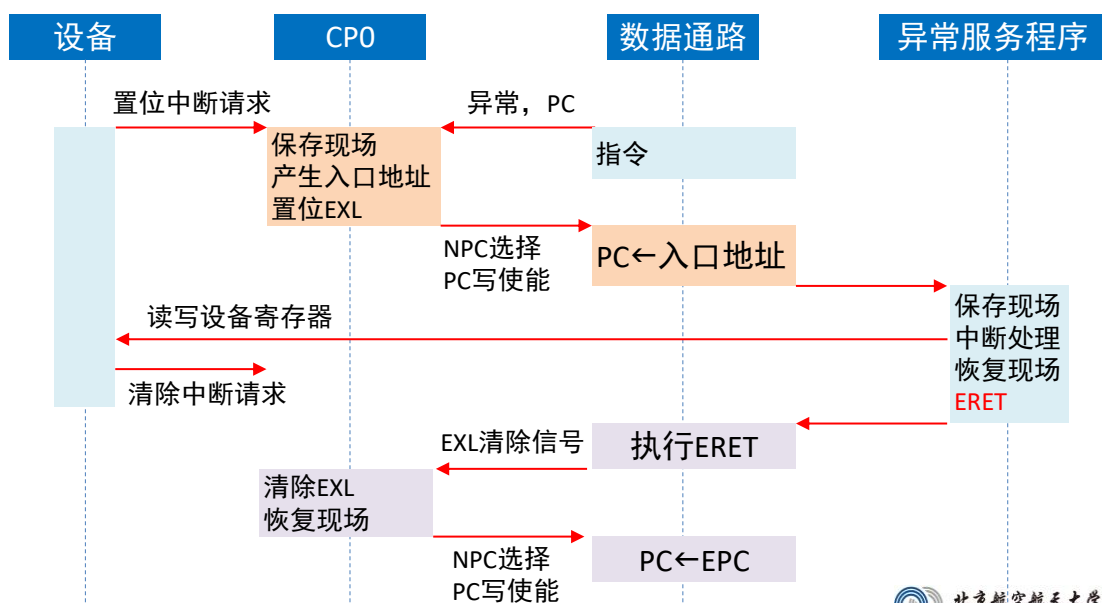
```
pc_exc = hwint ? PC+4 : PC
```



设计CP0：模块接口

信号名	方向	用途	产生来源及机制
A1[4:0]	I	读CP0寄存器编号	执行MFC0指令时产生
A2[4:0]	I	写CP0寄存器编号	执行MTC0指令时产生
DIn[31:0]	I	CP0寄存器的写入数据	执行MTC0指令时产生；数据来自GPR
PC[31:2]	I	中断/异常时的PC	PC
ExcCode[6:2]	I	中断/异常的类型	异常功能部件
HWInt[5:0]	I	6个设备中断	外部硬件设备(如鼠标、键盘)
We	I	CP0寄存器写使能	执行MTC0指令时产生
EXLSet	I	用于置位SR的EXL(EXL为1)	流水线在M阶段产生
EXLClr	I	用于清除SR的EXL(EXL为0)	执行ERET指令时产生
clk	I	时钟	
rst	I	复位	
IntReq	O	中断请求，输出至CPU控制器	是HWInt/IM/EXL/IM的函数
EPC[31:2]	O	EPC寄存器输出至NPC	
DOut[31:0]	O	CP0寄存器的输出数据	执行MFC0指令时产生，输出数据至GPR

中断响应机制分析：软硬件协同

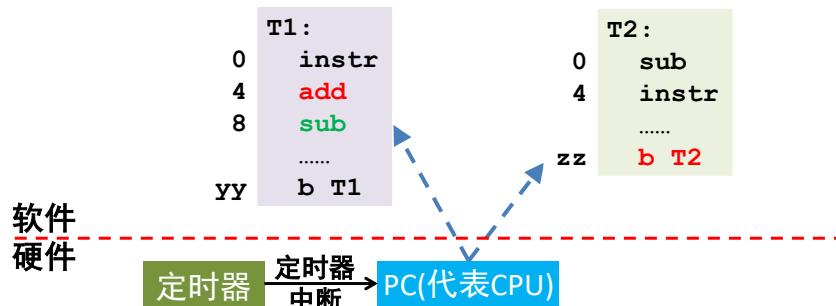


提纲

- 程序轮询I/O
- 异常/中断
- 支持异常/中断
- 协处理器
- 多任务切换机制

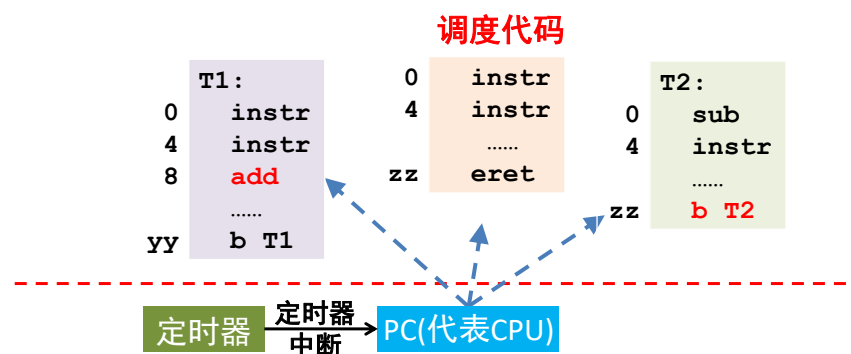
需求描述

- 场景：T1和T2两个任务（均为无限循环）定时切换
 - ◆ 定时器定时产生中断（这就是时间片）
 - ◆ 中断产生后（时间片到了），如果PC当前在执行T1则切换至T2，反之亦然
- 切换的具体说明（以从T1切换至T2为例）：
 - ◆ 时间片到时，PC从指向T1的add被修改为指向T2的b，然后开始执行T2
 - ◆ 当下个时间片到来时，PC要恢复执行T1的add的后继指令（sub）



任务切换必须依赖专用的调度代码

- 分析：系统中必须有独立的任务调度代码（第3段代码）
 - ◆ 这段代码必然与定时器中断服务程序相关（不妨假设就是）



关键在于如何保存和恢复不同任务的现场

- Q: 如何在调度代码(中断服务程序)中实现T1与T2间的切换?
- H: 关键要素在于利用EPC
 - EPC是任务切换的核心所在 (其他寄存器都是次要因素)

